Autonomous Pick-and-Place with Franka Research 3 (FR3) Robot

Industrial Robotics Project — A.Y. 2025/2026

Abstract

This report presents an autonomous pick-and-place system for the Franka Research 3 (FR3) manipulator, developed in ROS 2 Humble with MoveIt 2 and Gazebo (Ignition). Starting from a single launch command, the robot detects a red cube through an RGB-D perception pipeline, estimates its 3D pose, plans a collision-free trajectory to grasp it, transports it over an obstacle, and places it on a target. Beyond the baseline task, the system is organised around a *Behavior Tree* architecture that adds sensing-based grasp verification and a fail-safe recovery branch, is parametrised for deployment on the real robot, and has been evaluated quantitatively through an automated headless benchmark.

Contents

1 Introduction

The goal of this project was to build a fully autonomous pick-and-place system using the Franka Research 3 (FR3) robot inside a Gazebo simulation. The idea was to go from camera input to a completed manipulation task with no human intervention once the system is launched, using ROS 2 Humble, MoveIt 2 for the motion planning part, and an RGB-D camera for perception.

Concretely, the robot needs to do four things:

- Find the red cube in the scene using the camera.
- Plan a trajectory to reach and grasp it without hitting anything.
- Carry it over an obstacle to a target location.
- Put it down in the right place.

The whole pipeline runs from a single launch command and does not require any manual step in between.

Beyond this baseline behaviour, the system was progressively made more robust and closer to a real-robot deployment. The main additions, described in the following chapters, are: a *Behavior Tree* (BT) task architecture with an explicit fail-safe recovery branch (Chapter 4); sensing-based verification of the grasp; a fully parametrised planning scene so the same code can be adapted to a real laboratory setup without recompiling; and an automated headless benchmark that measures the success rate of the task over many independent runs (Chapter 6).

The project is organised into two ROS 2 packages: `cube_detector` (Python) for perception, and `cube_planner` (C++) for task and motion planning. The two communicate through a single topic, `/cube_pose`, keeping perception and planning cleanly decoupled.

Source code. The full source code is available at:
https://github.com/duccioneri3-pixel/franka-arm-ros-neri-duccio

2 Vision System Design

2.1 Chosen Approach: HSV Segmentation + RGB-D Pose Estimation

The vision system is implemented in the `cube_detector` ROS 2 package (Python). It uses a classical color-based approach combining HSV segmentation on the RGB image with metric 3D information extracted from the point cloud published by the simulated RGB-D camera.

The camera provides two inputs:

- An RGB image on topic `/fr3/depth_camera/image`.
- A point cloud on topic `/fr3/depth_camera/points`.

2.2 How It Works

Step 1 — HSV Segmentation. Each incoming RGB frame is converted from BGR to HSV color space. A binary mask is generated by thresholding the hue, saturation, and value channels to isolate red-colored pixels corresponding to the cube. Red straddles the hue wrap-around in HSV (near 0° and 180°), so two threshold ranges are combined with a bitwise OR to produce a robust mask:

```
lower_red1 = [0,   50, 50]      upper_red1 = [15,  255, 255]
lower_red2 = [155, 50, 50]      upper_red2 = [180, 255, 255]
```

Step 2 — Finding the Cube in 2D. From the binary mask, contours are extracted and the largest one is selected — that should be the cube. Contours with area below 100 pixels are discarded as noise. The bounding rectangle of the selected contour gives the 2D pixel region of the cube, and its centroid (c_x, c_y) is computed.

Step 3 — Getting the 3D Position (spatial robustness). The bounding-box centroid pixel (u, v) is used to look up the corresponding 3D point in the organized point cloud. Instead of reading a single pixel — which is fragile, since that exact point may be NaN due to sensor noise or surface reflections — the detector samples a small square window around the centroid (± 4 pixels) and takes the *median* of all valid points:

```
for each (uu, vv) in window around (u, v):
    idx = vv * width + uu
    if point[idx] is not NaN:
        collect point[idx]
position = median(collected points)
```

Using the median over a window makes the estimate robust to outliers and missing depth values, so a single bad pixel does not invalidate the frame.

Step 4 — Temporal Filtering (temporal robustness). On top of the spatial median above, the detector applies a *temporal median filter* on the published pose. The last N pose estimates (in the world frame) are stored in a fixed-length buffer, and the component-wise median of the buffer is published rather than the instantaneous reading:

```
buffer.append([x, y, z])          # last N poses (default N = 5)
published_pose = median(buffer)   # component-wise
```

This smooths the small frame-to-frame oscillations of the estimate and, more importantly, rejects isolated outlier readings: a single spurious detection does not move the median, so the planner always receives a stable and reliable pose. The window size N is a ROS 2 parameter (`pose_filter_window`), so the amount of smoothing can be tuned. Since the cube is static until it is grasped, the small latency introduced by the filter is irrelevant.

Step 5 — Frame Transformation and Publishing. The filtered 3D position is transformed from the camera frame to the world TF frame using the ROS 2 `tf2` library. The resulting pose is published as a `geometry_msgs/PoseStamped` message on the `/cube_pose` topic, where it is consumed by the `cube_planner` node.

Detection gating. To keep the logs and the planning scene clean, the detector can be enabled or disabled at runtime through a boolean topic, `/detector_enable`. Once the cube has been grasped it no longer makes sense to search for it on the table, so the planner disables the detector at that point (see Chapter 5).

2.3 Justification of the Chosen Method

For this specific task, a simple approach like this is more than enough. The cube is the only red object in the scene and the Gazebo lighting is uniform, so the HSV segmentation works reliably. Sampling a window around the centroid and taking the median — both spatially and, now, temporally — is fast and straightforward, and in a clean simulation environment it gives a reliable estimate of the cube position while remaining robust to occasional invalid depth pixels and transient outliers. More complex methods such as point-cloud clustering or a learning-based detector would only add unnecessary complexity here. For unstructured scenes or objects that are not known in advance, a learning-based perception model would become relevant; this is noted as a possible future direction in Chapter 7.

3 Task and Motion Planning Design

3.1 Overview

The motion planning side is handled by the `cube_planner` package, written in C++. It uses MoveIt 2 to plan and execute arm movements, and manages the collision scene so that the robot avoids obstacles automatically. The end-effector frame is `fr3_hand_tcp`, the planning group is `fr3_arm`, and the planning frame is `world`.

3.2 Setting Up the Planning Scene

Before any motion is planned, the planner populates the MoveIt planning scene with all relevant collision geometry:

- **Table:** a flat box covering the floor area, to prevent the arm from planning trajectories that would collide with the floor.
- **Obstacle:** a box (nominally $0.5 \times 0.2 \times 0.40$ m) placed at its known position in the world.
- **Cube:** a $0.04 \times 0.04 \times 0.04$ m box placed at the pose received from `/cube_pose`.

Once these are in the scene, MoveIt automatically avoids them when planning any trajectory.

Parametrised scene for sim/real adaptability. Rather than hard-coding the dimensions and positions of the obstacle and the table, these are exposed as ROS 2 parameters (arrays for dimensions and positions, plus the cube size). Their defaults reproduce the simulation scene, but they can be overridden — for example through a YAML file such as `scene_real.yaml` — with the measured layout of a real laboratory, *without recompiling*. Only the cube *position* always comes from the detector; everything else that describes the fixed environment is configurable. This makes the same planner directly reusable when moving from simulation to the physical robot.

3.3 Pick-and-Place Sequence

The full task is a sequence of elementary steps. A key design choice concerns *which motions are Cartesian (straight-line) and which are free-space (OMPL)*:

- **Cartesian** motions are used wherever a clean, deterministic straight-line path is desired — typically vertical descents and lifts.
- **OMPL** (RRTConnect) free-space planning is used wherever the arm must navigate around the obstacle, where a straight line would be invalid.

The steps are:

1. Open gripper. Both finger joints (`fr3_finger_joint1/2`) are opened before approaching.

2. Pre-grasp. The arm moves to a point 0.12 m above the cube. This aligns the end-effector for a top-down grasp and ensures the approach does not collide with the cube. (OMPL.)

3. Approach (Cartesian). A Cartesian path makes the arm descend straight down onto the cube, to $c_z - 0.015$ m. This vertical offset was an important tuning point: an earlier, deeper value (-0.035 m) placed the TCP only a few millimetres above the table, putting the goal in a near-collision configuration where the planner could not sample valid states. Raising the grasp to -0.015 m places the fingers correctly around the cube while staying clear of the table, which made both the approach and the subsequent lift reliable. If the Cartesian path cannot be completed for a configurable minimum fraction of the way, the planner falls back to OMPL toward the same target.

4. Grasp. The gripper is closed using the `follow_joint_trajectory` action.

5. Attach and verify. The cube is attached to the `fr3_hand` link in the MoveIt scene (with touch links `fr3_hand`, `fr3_leftfinger`, `fr3_rightfinger`) so that subsequent planning treats it as part of the robot. The grasp is then *verified* using the measured gripper width (Chapter 4).

6. Lift (Cartesian). The arm lifts straight up to $c_z + 0.35$ m.

7. Transport to target (OMPL). The arm plans a trajectory to a pose above the target. This is a free-space motion: OMPL automatically finds a path that avoids the obstacle, so some curvature of the trajectory here is expected and desirable — it is the planner going around the obstacle.

8. Place (Cartesian). Once above the target, the arm descends *straight down* to the release height. Making this a Cartesian vertical descent (rather than OMPL) avoids unnecessary sideways detours in the now obstacle-free zone above the target.

9. Release. The cube is detached and removed from the planning scene, and the gripper opens. Removing the cube object from the scene (not just detaching it) is necessary so that the following upward and retreat motions are not blocked by a phantom collision object left between the fingers.

10. Post-release lift (Cartesian). The arm rises straight up from the release pose, symmetric to the Place descent.

11. Retreat. Finally the arm moves clear of the placed cube.

3.4 Obstacle Avoidance

Obstacle avoidance is handled entirely through the MoveIt planning scene mechanism. The obstacle is inserted as a `CollisionObject`. The OMPL planner samples the configuration space and discards samples that result in collisions with scene objects, producing collision-free paths. Once the cube is attached to the gripper (Step 5), MoveIt also accounts for its volume during the transport and place motions, so the planner avoids collisions not only of the arm but also of the carried cube with the obstacle and the environment.

4 Behavior Tree Architecture

4.1 Motivation

A plain sequential script executes the pick-and-place steps one after another and aborts on the first failure, leaving the robot in an arbitrary state. For an autonomous system this is fragile: a single failed motion plan (which is always possible with a probabilistic planner such as RRTConnect) would leave the task half-done with no defined outcome. To make the behaviour robust and predictable, the task is structured as a *Behavior Tree* (BT), built with BehaviorTree.CPP (v4). The BT makes the control flow explicit and, crucially, adds a well-defined *recovery* behaviour.

4.2 Tree Structure

The core of the tree is a `Fallback` node whose first child is the main pick-and-place `Sequence` and whose second child is a recovery branch:

```
Fallback
  |-- Sequence  (main pick-and-place)
  |      OpenGripper -> PreGrasp -> Approach -> CloseGripper
  |      -> AttachCube -> CheckGrasp -> Lift -> Transport
  |      -> Place -> DetachCube -> OpenGripper -> ReleaseLift -> Retreat
  |
  |-- Sequence  (recovery / fail-safe)
        DetachCube -> OpenGripper -> GoHome
```

The semantics of the `Fallback` are: try the main sequence; if any step fails, the sequence returns `FAILURE` and the fallback then runs the recovery branch. The recovery releases the cube (both physically and from the planning scene) and drives the arm back to a known, safe "home" pose. This guarantees that, whatever happens during the task, the robot ends in a defined and safe configuration rather than stuck in mid-motion.

4.3 Sensing-Based Grasp Verification

After the cube is attached, a dedicated `CheckGrasp` node verifies that the grasp actually happened, instead of blindly assuming success. The node subscribes to `/joint_states` and reads the position of the two finger joints. Because the order of joints in the message is not guaranteed, the fingers are located *by name* (`fr3_finger_joint1/2`) rather than by a fixed index. The total gripper width is the sum of the two finger half-displacements, and it is compared against the expected cube size within a configurable tolerance:

```
width = finger1 + finger2
if |width - cube_size| <= tolerance:  grasp OK
else:                                 grasp failed -> recovery
```

If the check fails, the node returns `FAILURE`, which propagates up and triggers the recovery branch — so the robot does not lift and carry an empty gripper. Both the tolerance and an on/off switch are ROS 2 parameters. In simulation the measured width closely follows the commanded finger position, so the check is most discriminative on the real robot; nonetheless the sensing logic is correct and in place, ready for hardware.

4.4 Robustness of Cartesian and Free-Space Motions

Two recurring, intermittent issues were traced to the timing and the execution tolerance of the simulated arm, and were addressed at their root:

- **"Start point deviates from current robot state."** Before each Cartesian motion, the arm is allowed to settle and the MoveIt start state is re-anchored to the current state (`setStartStateToCurrentState()`). In addition, MoveIt's execution start tolerance (`allowed_start_tolerance`) is relaxed from the very strict default of 0.01 to 0.05 rad, which accommodates the small residual motion of the simulated arm. This is a legitimate tuning for a dynamic simulated system; on the real robot a more conservative value would be used, so the value is kept configurable.

- **Occasional lift/approach failures.** These were ultimately caused by the grasp being too deep (Chapter 6 discusses the residual failure mode), and were largely removed by raising the grasp offset. Where a Cartesian motion still completes only partially, an acceptance threshold decides whether to accept the partial vertical motion or fall back to OMPL, avoiding an OMPL call that tends to fail on these goals.

4.5 Why the Recovery Matters

The recovery branch is not a cosmetic addition: as shown by the experimental evaluation (Chapter 6), when the main sequence occasionally fails to plan the approach, the recovery reliably returns the robot to a safe home pose. In other words, the system *degrades gracefully* instead of breaking — which, for an autonomous manipulator, is arguably a more important property than never failing at all.

5 Controller and System Implementation

5.1 ROS 2 Architecture

The system is built as two ROS 2 nodes communicating through a single topic, on top of the Franka FR3 simulation environment (Gazebo Ignition + MoveIt 2):

- The `cube_detector` node (Python) subscribes to the RGB image and the point cloud, estimates the cube pose, and publishes it on `/cube_pose`.
- The `cube_planner` node (C++) subscribes to `/cube_pose`, builds the MoveIt planning scene, and executes the pick-and-place task through `MoveGroupInterface`.

The stack is started from a single launch file. Two entry points are provided: `full_demo.launch.py` runs the linear planner, while `bt_demo.launch.py` runs the Behavior Tree version (Chapter 4). Each brings up Gazebo and MoveIt first, then the detector, then the planner.

5.2 Perception Pipeline

The detector subscribes to the camera topics using the `sensor_data` QoS profile (best-effort), the standard profile for high-rate sensor streams, ensuring compatibility with the camera publishers. Each RGB frame is segmented in HSV, the cube centroid is located in 2D, and its 3D position is recovered from the point cloud (spatial median over a small window), then temporally filtered, transformed into the world frame, and published on `/cube_pose`.

5.3 Motion Execution

The planner waits until a cube pose has been received before starting (it retries instead of assuming a fixed startup time). It then configures `MoveGroupInterface` for the `fr3_arm` group, setting the planning frame to `world`, the planning time, and conservative velocity/acceleration scaling. As described in Chapter 4, the sequence mixes Cartesian-interpolated motions for the vertical approach, lift, place and post-release lift (each with a fallback to OMPL if the Cartesian path cannot be completed), and OMPL free-space planning for the pre-grasp and transport motions, which must route around the obstacle.

5.4 Detector Gating and Clean Shutdown

After the grasp, the planner publishes on `/detector_enable` to disable the detector: with the cube held between the fingers it is pointless to keep searching for it on the table, and disabling it keeps the logs clean during transport and place. On shutdown, the detector guards its `rclpy` teardown so that a `Ctrl-C` does not produce a double-shutdown error, giving a cleaner exit.

5.5 Gripper Control and Holding the Cube

The gripper is controlled through the `follow_joint_trajectory` action exposed by the simulated gripper controller. Rather than waiting a fixed amount of time after sending a command, the controller waits for the action *result* before continuing. This makes grasping robust to a slow

simulation: the sequence proceeds only once the fingers have actually finished moving, instead of relying on a fixed delay that could expire too early when the simulation runs slowly. The action timeouts are set generously for this reason.

Holding the cube: friction vs. attachment. The simulated gripper only exposes a position-based trajectory action, not a force-based grasp. The cube is therefore *physically* held during transport by the friction between the fingers and the cube in the Gazebo model. The MoveIt `attachObject` call, by contrast, is used only for *planning*: it informs the planner that the cube is rigidly attached to the hand, so that collision checking during transport accounts for the carried cube. In other words, the attachment is not what physically holds the cube — friction is — it only keeps the planning model consistent. On the real Franka, a force-based grasp action would provide the physical holding directly; the corresponding code path is already structured for that case.

6 Experimental Evaluation

6.1 Goal and Methodology

To evaluate the system quantitatively rather than anecdotally, the complete pick-and-place task was executed many times under identical conditions and the outcome of each run was recorded automatically. Because RRTConnect is a probabilistic planner, individual runs are not identical, so a success rate over many independent runs is a more meaningful measure than a single demonstration.

Headless execution. Running dozens of trials with the full graphical stack is impractical. However, in Ignition Gazebo the sensor rendering (needed by the RGB-D camera) depends on a rendering context, and a pure server-only launch (`-s`) crashed on this WSL setup with an Ogre rendering exception. The solution was to run the normal (GUI) launch inside a *virtual framebuffer* (`xvfb-run`), which provides an off-screen rendering context without a physical display. This is a standard technique for headless rendering and does not alter the simulation: physics, camera rendering, planning and execution are identical to a run with a visible window — only the pixels are drawn to an in-memory buffer. This makes automated, unattended batch runs possible.

Orchestration. A benchmark script launches the task headless N times. For each run it captures the log, waits for the final verdict (or a timeout), classifies the outcome, aggressively cleans up all processes to avoid contamination between runs, and appends the result to a CSV file. Each run is classified as:

- **Direct success:** the pick-and-place completed normally, the cube was grasped, transported and placed.
- **Recovered:** a step failed (e.g. the approach could not be planned), the recovery branch ran and returned the robot to the home pose. The robot ends safe, but the cube is *not* moved.
- **Timeout / crash:** the run did not reach a verdict within the time limit (an infrastructure issue, not a task outcome).

6.2 Results

A batch of 35 runs was executed. Each run took on the order of 60–120 seconds. Table 1 summarises the outcomes.

Outcome	Count	Percentage
Direct success (task completed)	27	77.1%
Recovered (task failed, robot safe)	7	20.0%
Timeout / crash (infrastructure)	1	2.9%
Total	**35**	**100%**

Table 1: Outcome of 35 independent headless runs.

6.3 Discussion

Two honest conclusions follow from these numbers.

First, the **real task success rate** — the cube actually picked and placed — is $27/35 \approx 77\%$. This is a realistic figure for manipulation with a sampling-based planner: it is not, and should not be expected to be, 100%. It would be misleading to report the 34/35 figure (direct plus recovered) as "success", because in the 7 recovered runs the cube was never moved; the run simply ended safely. Inspecting the logs of these runs confirms the failure mode: the Cartesian *approach* occasionally completes only partially (e.g. 37%), the OMPL fallback is then unable to sample valid states for the goal, and the recovery branch is triggered.

Second, and more positively, the **recovery worked in every failure case** (7/7): whenever the task failed, the robot was returned to a safe home pose rather than left stuck or making unsafe motions. This is the practical value of the Behavior Tree architecture with an explicit fallback: the system degrades gracefully. The single timeout run was an infrastructure-level stall of the headless environment (the run hung far beyond the per-run limit), not a failure of the planning or perception, and is reported separately for transparency.

Actionable takeaway. The evaluation localises the weak point precisely: the intermittent approach failure. This is a concrete, addressable target for future work (for example, further tuning of the approach waypoints or the planner configuration), rather than a vague "sometimes it fails".

A note on acceleration limits. During execution MoveIt reports that joint acceleration limits are not defined in the robot's `joint_limits.yaml` and falls back to a default of 1 rad/s^2. This file belongs to the upstream robot description and was intentionally left unmodified; the default behaviour is adequate for the task. This is noted here as a conscious choice rather than an oversight.

7 Conclusions and Future Work

7.1 Summary

The system performs the full pick-and-place task autonomously: from a single launch command, the robot detects the red cube, estimates its 3D pose, plans a collision-free trajectory, grasps the cube, transports it over the obstacle, and places it on the target, with no manual intervention. Beyond the baseline task, the work delivered a more mature and robust system:

- a **Behavior Tree architecture** with an explicit fail-safe recovery branch, so the robot always ends in a defined, safe state;
- **sensing-based grasp verification** from the measured gripper width;
- a **spatially and temporally filtered** perception pipeline;
- a **fully parametrised planning scene**, decoupling the code from the specific simulation layout and preparing it for real-robot deployment;

- straight-line **Cartesian motions** for the vertical approach/lift/place phases, removing unnecessary detours;
- an **automated headless benchmark** that measures the task success rate honestly over many runs.

7.2 Results and Challenges

The HSV + RGB-D perception reliably locates the cube, and the MoveIt-based planner produces collision-free trajectories that avoid both the obstacle and the carried cube. Over 35 independent runs, the task completed successfully in about 77% of cases, and in every failure the recovery returned the robot safely home (Chapter 6).

The main practical challenges were related to timing and to the simulated gripper. Early versions relied on fixed delays, which caused the grasp to fail when the simulation ran slowly; this was solved by waiting for the action result. Holding the cube during transport required tuning the friction in the Gazebo model, since the simulated gripper exposes only a position-based action and not a force-based grasp. Finally, several intermittent planning failures were traced to a grasp pose that was too deep (placing the goal near a collision) and to an overly strict execution start tolerance; both were addressed at their root.

7.3 Future Work

- **Real-robot deployment.** The most significant next step is to run on the physical Franka FR3, using the force-based `franka_gripper` Grasp action instead of the position-based trajectory, which would remove the dependency on simulated friction. The parametrised scene and the already-structured real-gripper code path are steps toward this.
- **Reducing the approach failure rate.** The evaluation localised the residual failure to the approach motion; tuning the approach waypoints or the planner configuration is a concrete improvement.
- **Camera calibration.** An eye-to-hand calibration step would be required for perception on real hardware.
- **Perception for unstructured scenes.** For objects not known in advance or cluttered scenes, a learning-based perception model could replace the color-based detector; this is unnecessary for the present controlled task but would extend the system's applicability.